

DEVOPS PIPELINE'LARINDA API REGRESYON TESTLERİ VE AI DESTEKLİ TEST MÜHENDİSLİĞİ

Levent Kutay Sezer
22360859013

Yazılım Test mühendisliği - proje ödevi

github.com/Kutibios/testmuh

SUNUM AKIŞI :

- 1. Yazılım Test Mühendisliği nedir, neden var?**
- 2. Test piramidi ve API testlerinin yeri**
- 3. Regresyon testi ve otomasyon zorunluluğu**
- 4. DevOps & CI/CD pipeline'da testin konumu**
- 5. Rest Assured — Java ekosisteminin standardı**
- 6. AI Destekli Test Mühendisliği — bugünkü manzara**
- 7. Proje Demo — kod + canlı çalıştırma**

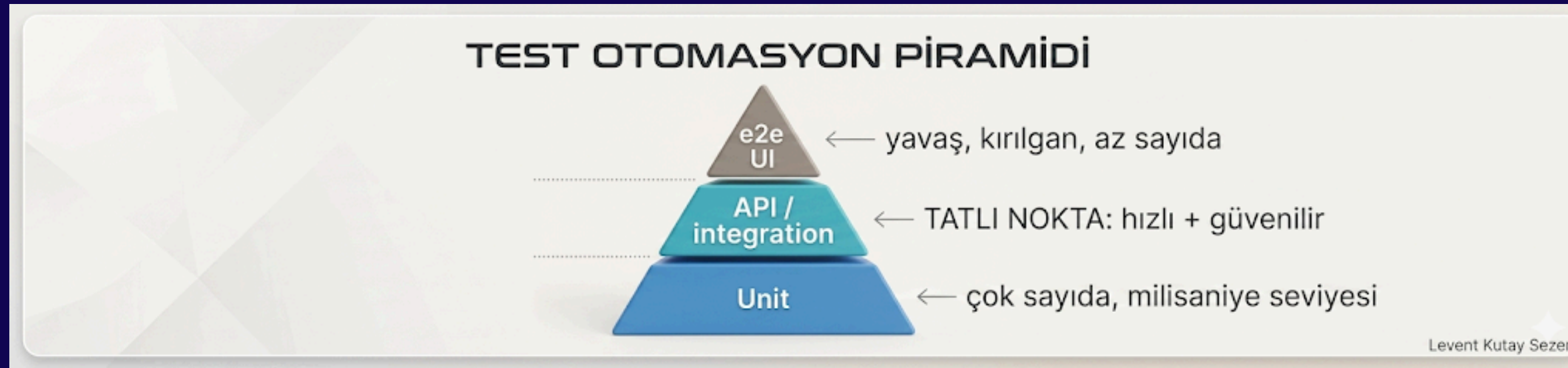
YAZILIM TEST MÜHENDİSLİĞİ NEDİR?

Yazılımın beklenen davranışı sergilediğini ve beklenmeyen davranışlardan kaçındığını sistematik biçimde doğrulayan disiplin.

Neden var?

- **Yazılım karmaşıklığı her sürümde artar**
- **Hatanın geç bulunması, erken bulunmasından 10-100 kat daha pahalıdır**
- **Manuel test ölçeklenmez — sürekli teslimat (CI/CD) testin otomatize olmasını zorunlu kılar**

TEST PİRAMİDİ



- **Unit:** tek fonksiyon, izole, ms cinsinden
- **Integration / API:** servisler arası sözleşme
- **End-to-End (UI):** tüm sistem, en pahalısı

API testleri, maliyet/değer oranı en yüksek olan katmandır.

REGRESYON TESTİ NEDİR

Regresyon = "Geriye Gitme" Halihazırda stabil çalışan bir özelliğin, sisteme eklenen yeni bir kod değişikliği nedeniyle beklenmedik şekilde bozulması durumu.

- Her yeni özellik (feature), mevcut çalışan yapıyı istemeden kırabilir.
- Her yeni sürümde tüm sistemi manuel olarak test etmek zamanen imkansızdır.
- Otomatik Regresyon Seti (Suite) = Her entegrasyonda (commit) devreye giren dijital bir güvenlik ağıdır.

İdeal bir regresyon test seti nasıl olmalı?

- Hızlı ve Çevik: Geliştirme süreçlerini (developer workflow) asla yavaşlatmamalı.
- Kararlı ve Tutarlı (Deterministic): Rastgele başarısızlıklar (flaky sonuçlar) üretmemeli.
- Odaklanmış: Her uç senaryoyu (edge case) değil, kritik iş akışlarını kapsamalı.

DevOps & CI/CD'deki Yeri

Geliştirici → commit → CI Pipeline

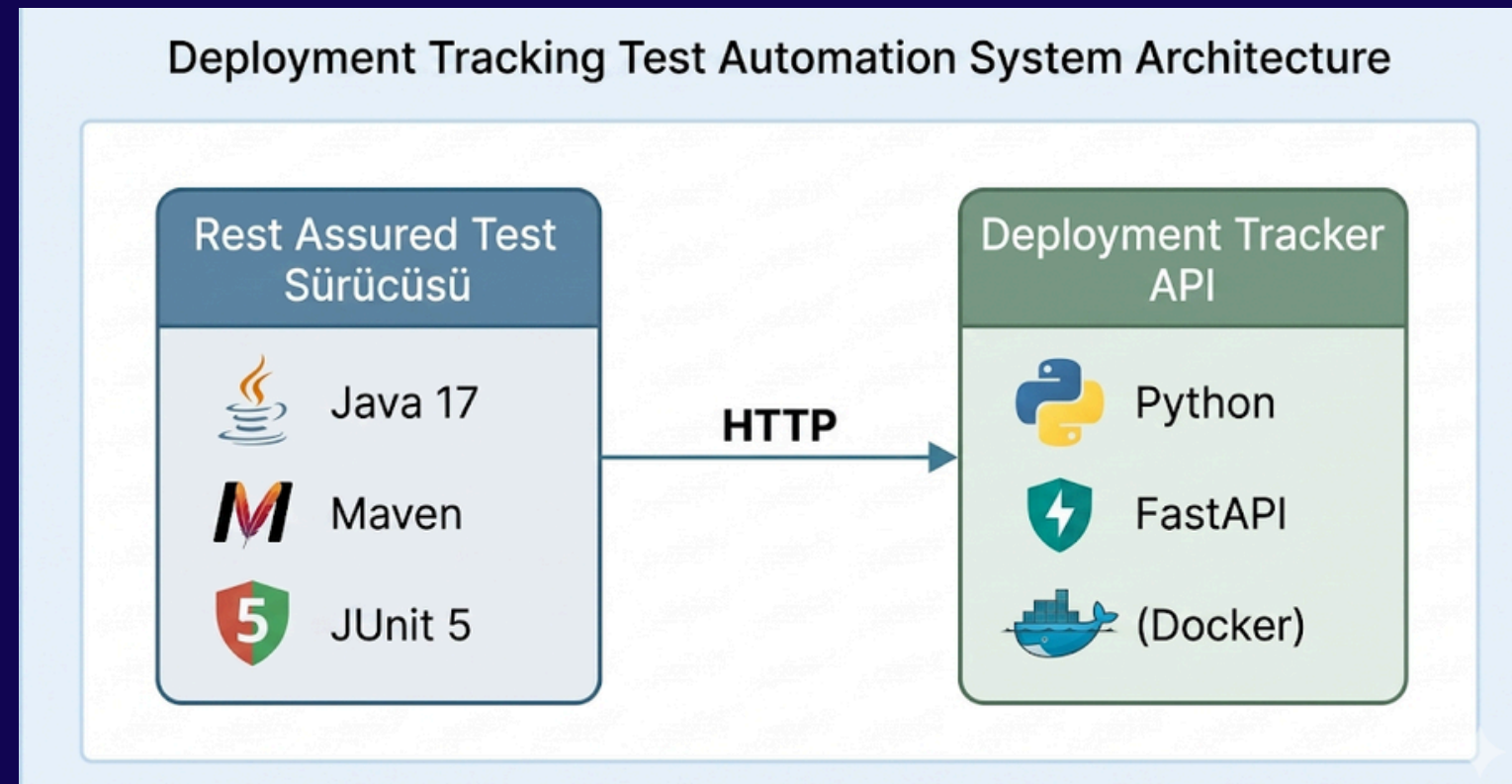
- 1. Lint / Static Analysis
- 2. Unit Tests (saniyeler)
- 3. Build / Package
- 4. Container Image
- 5. API/Integration Tests ← BURADAYIZ
- 6. Deploy to staging
- 7. E2E Smoke Tests

Anahtar Kavramlar:

- Shift-left: testi geliştiriciye en yakın yere taşı
- Fail-fast: hatalı build canlıya gitmesin
- Pipeline gating: testler yeşil değilse merge yok

Containerized Testing

Test edilen servis ve test kořucusu bağımsız
container'lar halinde:



Avantajları:

- Her ortamda aynı şekilde çalışır (laptop = CI = staging)
- Servisi `docker compose up` ile tek komutta ayağa kaldırırız
- Testler `localhost:8002`'e konuşur — gerçek HTTP, mock değil

Rest Assured Nedir?

Java için fiili standart REST API test kütüphanesi. BDD-tarzı okunabilir sözdizimi:

```
given()
    .contentType(JSON)
    .body(yeniDeployment)
.when()
    .post("/deployments")
.then()
    .statusCode(201)
    .time(lessThan(2000L))
    .body("service", equalTo("payment-api"))
    .body("id", notNullValue());
```

- HTTP isteği, doğrulama, raporlama → ****tek bir akış****
- JUnit, TestNG, Maven, Gradle ile sorunsuz entegre
- JSON Path + Hamcrest matchers ile esnek body doğrulama

GET Testleri — DeploymentGetTest.java

```
@Test
@DisplayName("GET /deployments -> 200 OK, JSON array, 2 saniye altinda")
public void deployment_listesi_alinabilmeli() {
    given()
        .spec(requestSpec)
    .when()
        .get("/deployments")
    .then()
        .spec(responseSpec)
        .statusCode(200)
        .time(lessThan(2000L))
        .body("size()", greaterThanOrEqualTo(0));
}
```

deployment listesi alınabilmeli

GET /deployments endpoint'ine istek atılır. Üç kontrol yapılır: HTTP status kodu 200 olmalı, cevap 2 saniyenin altında gelmeli, dönen veri bir JSON array olmalı. size() ifadesi Rest Assured'ın JSON Path sözdizimi — array uzunluğunu sorgular.

DELETE Testleri — DeploymentDeleteTest.java

```
@Test
@DisplayName("DELETE /deployments/{bilinmeyen-id} -> 404 (negatif)")
public void var_olmayan_deployment_silinmeye_calisilince_404_donmeli() {
    given()
        .spec(requestSpec)
    .when()
        .delete("/deployments/yok-boyle-id-99999")
    .then()
        .spec(responseSpec)
        .statusCode(404)
        .body("detail", containsString("Deployment bulunamadi"));
}
```

var olan deployment silinebilmeli

Önce POST ile yeni bir kayıt oluşturulur, dönen id alınır. Sonra o id ile DELETE isteği atılır. Beklenen: 204 No Content (silme başarılı, body yok) ve 2 saniye altı response time. Her test kendi kaydını oluşturup kendi siliyor. Bu bir test bağımsızlığı ilkesidir.

POST Testleri — DeploymentPostTest.java

```
@Test
@DisplayName("POST /deployments -> 201 Created, body alanlari dogru, 2 saniye altinda")
public void yeni_deployment_kaydedilebilmeli() {
    given()
        .spec(requestSpec)
        .body(YENI_DEPLOYMENT_BODY)
    .when()
        .post("/deployments")
    .then()
        .spec(responseSpec)
        .statusCode(201)
        .time(lessThan(2000L))
        .body("id", notNullValue())
        .body("service", equalTo("payment-api"))
        .body("version", equalTo("v1.4.2"))
        .body("environment", equalTo("prod"))
        .body("status", equalTo("in_progress"))
        .body("created_at", notNullValue());
}
```

eklenen deployment get ile doğrulanabilmeli

Önce POST yapılır ve dönen id yakalanır. Sonra aynı id ile GET yapılır. Bu bir end-to-end zincir: servis "201 döndüm" deyip kaydı aslında saklamamış olabilir. Bu da bir test bunu ortaya çıkarır.

AI Destekli Test Mühendisliği

Test mühendisliğinde AI artık deneysel değil, **takım üyesi**

Alan	Klasik	AI Destekli
Test case yazma	Mühendis spec'i okur, elle yazar	LLM spec'ten taslak üretir, mühendis düzeltir
Locator'lar (UI)	Sabit selector, kırılgan	Self-healing — DOM değişince adapte olur
Test verisi	Manuel veya factory	Üretici modeller ile sınır/edge senaryo
Önceliklendirme	"Hep aynı suite"	Risk-bazlı: değişen kodu vuran testleri öne al
Hata triyajı	Mühendis log okur	LLM log+diff özetler, kök neden önerir

SELF HEALING TESTS

UI/API şeması değişti, test eski locator/alani arıyor → klasik suite kırmızı.

Self-healing araç (Mabl, Testim, Functionize):

1. Eski locator/alani bulamadı
2. Benzerlik (semantic + DOM) ile muhtemel yeni karşılığı buldu
3. Testi kendi başına güncelledi, mühendise notify etti

Risk: otomatik onarım yanlış olabilir → mutlaka **insan onayı**.

Bu yüzden AI'ı **tamamlayıcı** olarak görmek lazım, **ikame** olarak değil.

PROJE-DİZİN YAPISI

```
testmuh/
├── docker-compose.yml
├── service/                                # FastAPI mini-servis
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/main.py, store.py
├── tests/                                  # Java/Maven test projesi
│   ├── pom.xml
│   └── src/test/java/com/testmuh/deployments/
│       ├── BaseTest.java
│       ├── HealthTest.java
│       ├── DeploymentGetTest.java
│       ├── DeploymentPostTest.java
│       └── DeploymentDeleteTest.java
```

Riskler & Sınırlar

- Halüsinasyon — LLM olmayan bir alan/endpoint uydurabilir
- False positive / negative — test geçti diye ürün doğru çalışıyor demek değil
- Aşırı güven — "AI yazdı, kontrol etmedim" = canlıda bug
- Veri gizliliği — kapalı kaynak/üretim verisi LLM'e gönderilmemeli

TEŞEKKÜRLER

GitHub Repo:
github.com/Kutibios/testmuh